

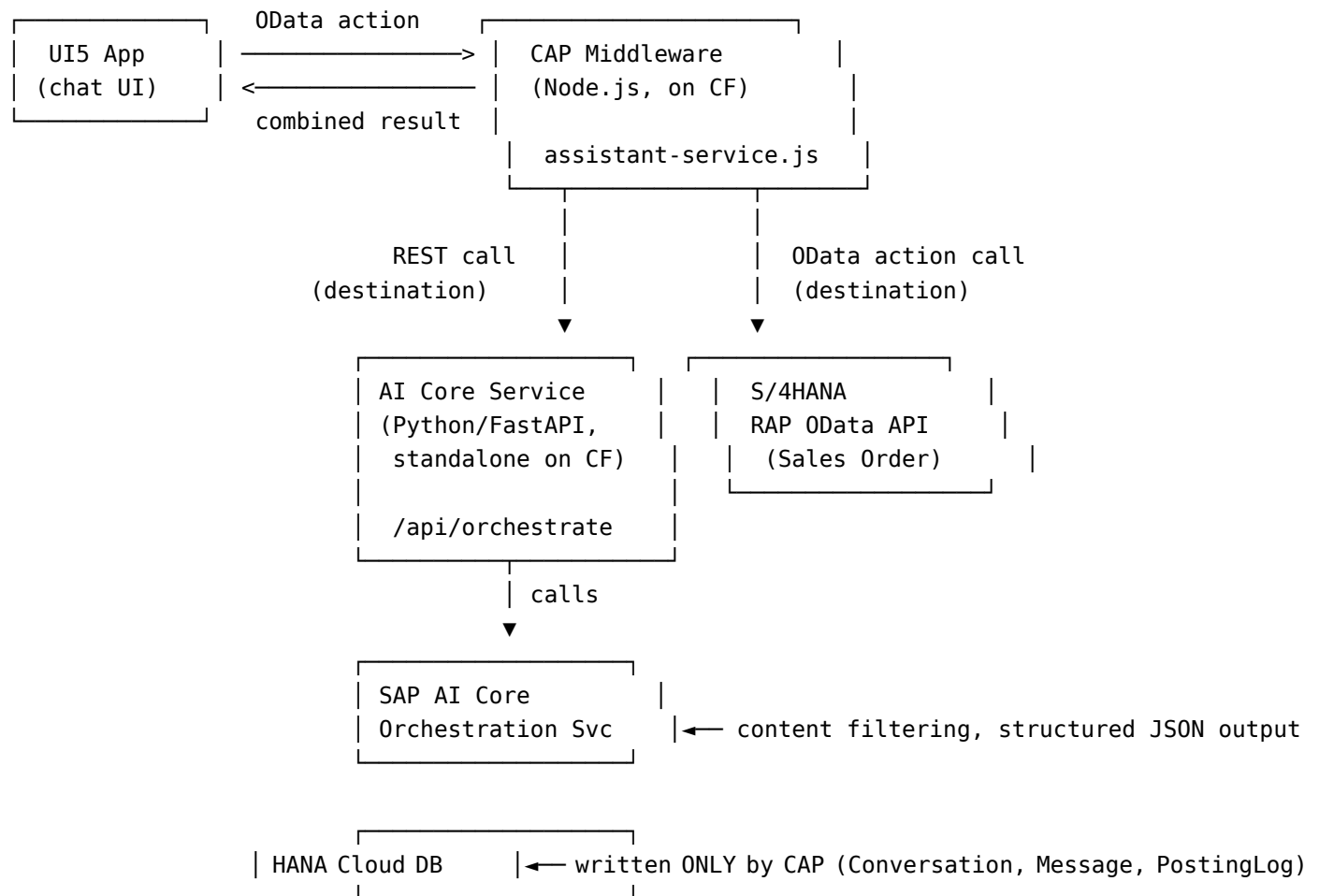
AI Sales Assistant — End-to-End Architecture Walkthrough

A simple, complete reference project showing how SAP AI Core, CAP, S/4HANA, HANA Cloud, and UI5 fit together on Cloud Foundry. One user question flows through every layer of a realistic SAP AI project — this document follows that single flow from click to database, step by step.

The Scenario

A user types “Create a sales order for customer ACME, 10 units of material M-100” (or just asks a plain question) into a UI5 chat app. The AI figures out whether this is a question or a posting request. If it’s a posting request, the system actually creates a Sales Order in S/4HANA. Every turn of the conversation is saved in HANA Cloud.

Architecture at a Glance



Three deployable units on Cloud Foundry: the **UI5 app + approuter + CAP service** (one MTA), the **Python AI Core service** (its own cf push), and **HANA Cloud** (a bound service, not a CF app). Two BTP **Destinations** connect them — CAP never hardcodes a URL.

Folder Structure

```
sample_project_capm_s4hana_ai/
├── ai_core_service/          <- Python FastAPI microservice (standalone CF app)
│   ├── main.py              <- mounts every router, /health endpoint
│   ├── schemas.py           <- Pydantic request/response models
│   ├── llm_client.py         <- cached SAP AI Core LLM + embedding client
│   ├── hana_db.py            <- read-only HANA connection (RAG + SQL routers)
│   └── routers/
```

```

| | | └─ chat.py                <- POST /api/chat
| | | └─ rag.py                 <- POST /api/rag-query
| | | └─ sql_query.py          <- POST /api/sql-query
| | | └─ orchestrate.py        <- POST /api/orchestrate (the one CAP actually calls)
| └─ requirements.txt, manifest.yml, .env.example
|
└─ capm_middleware/           <- CAP project (Node.js, deployed as an MTA)
  └─ db/schema.cds            <- Conversation, Message, PostingLog entities
  └─ srv/assistant-service.cds <- service contract: askAssistant action
  └─ srv/assistant-service.js  <- the orchestrator – calls AI Core + S/4HANA + HANA
  └─ app/assistant-ui/webapp/  <- the UI5 app (bundled CAP-style)
  └─ package.json              <- destinations declared under cds.requires
  └─ xs-security.json, mta.yaml

```

Step 1 — UI5 Sends the User’s Message

```

// app/assistant-ui/webapp/controller/App.controller.js
onSend: async function () {
  const sMessage = this.byId("messageInput").getValue().trim();
  const oModel = this.getView().getModel(); // OData V4 model, from
  ↪ manifest.json
  const oAction = oModel.bindContext("/askAssistant(...)"); // binds to the CAP action
  oAction.setParameter("conversationId", this._conversationId);
  oAction.setParameter("message", sMessage);
  await oAction.execute();
  const oResult = oAction.getBoundContext().getObject();
  this._conversationId = oResult.conversationId; // carry forward to the next turn
}

```

Key Points

- `bindContext(...).execute()` is UI5’s way of calling an OData V4 **action** (not just reading data) — the UI5 equivalent of calling `agent.ask(question)` in the earlier Python templates.
- The UI5 app calls **only** the CAP service — never SAP AI Core or S/4HANA directly. Credentials for those systems never reach the browser.
- `conversationId` is kept in a local JS variable and resent every turn — this is how the browser tells CAP “this is a continuation of an existing conversation,” matching the same “client resends what it wants remembered” pattern used throughout the AI Core sheets.

Step 2 — CAP Receives the Action (Service Contract)

```

// srv/assistant-service.cds
service AssistantService @(requires: 'authenticated-user') {
  action askAssistant(conversationId: String, message: String) returns AssistantResult;
}

```

Key Points

- `@(requires: 'authenticated-user')` means XSUAA must have already validated the caller’s JWT before this action runs — enforced by the framework, not hand-written auth-check code.
- Declaring the action’s input/output shape in CDS gives you free request validation and an

auto-generated OpenAPI/OData metadata document — the UI5 app's manifest.json reads this to know the action exists and what parameters it needs.

Step 3 — CAP Calls the Python AI Core Service

```
// srv/assistant-service.js
const aiCoreService = await cds.connect.to('AI_CORE_SERVICE'); // destination-backed, see
↪ package.json
// ...
const aiResult = await aiCoreService.send('POST', '/api/orchestrate', { message });
```

Key Points

- `cds.connect.to('AI_CORE_SERVICE')` resolves to a destination named in `package.json` (`cds.requires.AI_CORE_SERVICE`) — the actual URL lives in the BTP Destination service configured in `mta.yaml`, never in this JS file. Same principle as the Python templates' `.env/bound-service` pattern: **code reads a logical name, the platform resolves it to a real address.**
- If this call fails, `req.error(502, ...)` is returned — the whole action stops here rather than silently continuing with no AI answer.

Step 4 — Python AI Core Service: Orchestrated Chat + Intent Detection

```
# ai_core_service/routers/orchestrate.py
INTENT_SCHEMA = {
    "type": "object",
    "properties": {
        "answer": {"type": "string"},
        "is_posting_intent": {"type": "boolean"},
        "customer": {"type": "string"}, "material": {"type": "string"}, "quantity": {"type":
        ↪ "integer"},
    },
    "required": ["answer", "is_posting_intent", "customer", "material", "quantity"],
    "additionalProperties": False,
}

template = Template(
    messages=[SystemMessage("...determine if they want to CREATE a sales order..."),
    ↪ UserMessage("{?text}")] ,
    response_format=ResponseFormatJsonSchema(name="assistant_result", schema=INTENT_SCHEMA,
    ↪ strict=True),
)

config = OrchestrationConfig(template=template, llm=LLM(name="gpt-4o-mini"),
                             filtering=Filtering(input_filtering=..., output_filtering=...))
result = service.run(config=config, template_values=[TemplateValue(name="text",
    ↪ value=req.message)])
parsed = json.loads(result.orchestration_result.choices[0].message.content)
```

Key Points

- ONE LLM call does TWO jobs — answering AND classifying posting intent — because `strict`:

true JSON schema output **guarantees** the response has both an answer string and structured fields, safe to `json.loads()` directly with no parsing risk.

- Content filtering (input + output) runs as part of this same orchestration config — unsafe input never reaches the model, unsafe output never reaches CAP.
- This endpoint is **stateless** — it has no idea this is turn 3 of a conversation. Memory (if needed) would be added by CAP resending prior turns as `messages_history`, the same pattern covered in the Orchestration Advanced sheet.

Step 5 — CAP Persists the Conversation to HANA

```
// db/schema.cds
```

```
entity Conversation : cuid, managed { userID: String; title: String; to_messages: Composition of many Message
```

```
entity Message : cuid, managed { conversation: Association to Conversation; role: String; content: LargeStr
```

```
// srv/assistant-service.js
```

```
await INSERT.into(Message).entries([
  { ID: uuidv4(), conversation_ID: conversationId, role: 'user', content: message, mode:
  ↪ aiResult.mode },
  { ID: uuidv4(), conversation_ID: conversationId, role: 'assistant', content:
  ↪ aiResult.answer, mode: aiResult.mode },
]);
```

Key Points

- **This is the only place in the entire project that writes conversation data** — the Python AI Core service is deliberately kept stateless so persistence has a single owner, avoiding the classic bug of two services both trying to “own” the same data.
- `INSERT.into(...).entries([...])` inserting both the user AND assistant message in one call keeps a turn atomic — you never end up with a user message saved but no matching answer.
- This is CQL (sheet 09’s `SELECT.from/INSERT.into` pattern) — parameterized automatically, compiles to real HANA SQL under the hood, running against the HDI container declared in `mta.yaml`.

Step 6 — CAP Calls the S/4HANA RAP-Based OData Action

```
// srv/assistant-service.js
```

```
if (aiResult.is_posting_intent && aiResult.posting_payload) {
  const { customer, material, quantity } = aiResult.posting_payload;
  const s4Response = await s4hanaService.send('POST',
    '/sap/opu/odata4/sap/api_salesorder/srvc/sap/salesorder/0001/SalesOrder', {
    SoldToParty: customer,
    to_Item: { results: [{ Material: material, RequestedQuantity: quantity }] },
  });
  s4hanaOrderId = s4Response.SalesOrder;
}
```

Key Points

- **What RAP is:** the RESTful ABAP Programming Model is how S/4HANA exposes business objects (like Sales Order) as OData services, implemented in ABAP on the S/4HANA side (Behavior Definitions + Behavior Implementations). CAP does **not** write or deploy any ABAP

— it only calls an OData action that a RAP developer already published, exactly like calling any other external REST/OData API.

- s4hanaService is a second destination-backed connection (cds.requires.S4HANA_SALES_API) — same pattern as Step 3’s AI Core connection, different target system.
- This call is wrapped in try/catch — **a failed S/4HANA posting does not fail the whole request**. The user still gets their AI answer; postedToS4Hana just comes back false and a PostingLog row records the failure for follow-up.

Step 7 — CAP Returns One Combined Result

```
return { conversationId, answer: aiResult.answer, mode: aiResult.mode, postedToS4Hana,  
  ↪ s4hanaOrderId };
```

Key Points

- The UI5 app gets everything it needs in **one round trip** — the AI’s answer AND whether a real S/4HANA document was created — instead of needing a second call to check posting status.
- AssistantResult (the CDS type in assistant-service.cds) defines this exact shape, so UI5’s OData model knows what fields to expect without any manual JSON parsing.

Step 8 — UI5 Renders the Result

```
if (oResult.postedToS4Hana) {  
  MessageToast.show(`Posted to S/4HANA – Sales Order ${oResult.s4hanaOrderId}`);  
}
```

Key Points: the UI only reacts to fields on the already-typed AssistantResult — it never needs to know HOW the answer was produced or whether AI Core, HANA, or S/4HANA were involved. That’s the entire point of routing everything through one CAP action.

Deployment on Cloud Foundry

Unit	How it’s deployed	What it needs
ai_core_service	cf push using its own manifest.yml, python_buildpack	Bound AI Core service instance, bound HANA service instance
capm_middleware	cf build && cf deploy using mta.yaml (CAP srv + approuter + UI5 + HDI container)	XSUAA, Destination service (2 destinations), HDI container
HANA Cloud	Not a CF app — a bound service instance both units connect to	Provisioned once in BTP Cockpit

Why two separate deployables instead of one? The Python AI Core service uses a completely different buildpack/runtime (python_buildpack) than the CAP service (nodejs) — Cloud Foundry apps are single-runtime, so they must be separate cf push/MTA units even though they’re part of the same logical project. The **Destination service** is what lets CAP find the Python app’s route at runtime without either side hardcoding the other’s URL — redeploy either one independently and, as long as the destination config is updated, nothing else needs to

change.

Key Design Decisions (Interview-Relevant)

- **Stateless AI service, stateful CAP layer:** the Python service never writes to HANA for conversation data — only CAP does. Single owner of persistence avoids data-consistency bugs.
- **Destinations, not hardcoded URLs:** both outbound calls (AI Core service, S/4HANA) go through named destinations resolved by the platform — standard BTP integration pattern, makes moving between dev/test/prod trivial (same code, different destination config per landscape).
- **One action, one round trip:** askAssistant does AI call + persistence + conditional S/4HANA posting in one server-side action — the UI5 app doesn’t orchestrate multiple calls itself.
- **Posting failure is non-fatal:** an S/4HANA error doesn’t lose the AI’s answer or crash the conversation — it’s logged (PostingLog) and surfaced as postedToS4Hana: false.
- **Structured JSON output does double duty:** one LLM call returns both a human-readable answer and machine-readable posting fields — no second “extraction” call needed.

Quick Glossary

Term	Meaning in this project
RAP	RESTful ABAP Programming Model — how S/4HANA exposes business objects as OData APIs (ABAP-side, not built here).
Destination	BTP service that stores a target URL + auth so CAP code never hardcodes credentials.
HDI container	The isolated HANA schema this CAP app owns — where Conversation/Message/PostingLog actually live.
Approuter	Single entry point in production — serves the UI5 app, proxies OData calls to CAP, enforces XSUAA auth.
MTA	Multi-Target Application — one deployable bundle (srv + db-deployer + approuter + resources).
Orchestration Service	SAP AI Core’s managed pipeline (templating + filtering + structured output) called by orchestrate.py.